

ECE 370: Computer Organization and Assembly Programming

Lab 1 – Introduction to AVR Development Tools

SECTION OVERVIEW

The objectives of this lab are as follows:

For Part 1:

- Download and assemble the sample AVR assembly source code.
- Connect and operate the AVR programmer.
- Upload the previously-compiled sample program to the flash memory of the TekBots AVR microcontroller (ATmega32U4) board, and observe it running.

For Part 2:

- Download and compile the sample C program.
- Configure the I/O ports of the ATmega32U4 microcontroller.
- Write a simple C program for the ATmega32U4 microcontroller.
- Upload the code to your ATmega32U4 board and verify its correct operation.

You have two weeks (Weeks 1-2) to complete this lab.

PROCEDURE - PART 1

Compiling an AVR Assembly Program

1. Download the sample code (`BasicBumpBot.asm`) as well as the pre-compiler directive file (`m32U4def.inc`). This is a simple AVR assembly program that is well-commented and ready to assemble. All the codes that you develop for this course should be as well-commented as this code. Save this code to your favorite directory.
2. You are required to use a Windows system to develop your AVR assembly code throughout this course. You will be using it to write assembly programs for your AVR microcontroller board, which uses an ATmega32U4 microcontroller.

3. Now that you have loaded `BasicBumpBot.asm` and `m32U4def.inc` to the same directory, take a look at the sample code. Although you have not yet seen any AVR assembly code in this class, read the comments and see if you can follow the general structure and flow of the program. The general operation of the program is described in Figure 1.
4. Follow the installation guide and learn how an AVR assembly program is assembled. The resulting output is a binary program file (called a HEX file, with a `.hex` extension). This HEX file contains the actual binary instructions that will be executed by the ATmega32 microcontroller.

- Initialize key components of the ATmega32
- Start the TekBot moving forward
- Poll the whiskers for input
- If right whisker is hit:
 - Back up for a second
 - Turn left for a second
 - Continue Forward
- If left whisker is hit:
 - Back up for a second
 - Turn right for a second
 - Continue Forward

Figure 1: Operations for the `BasicBumpBot.asm` code.

Uploading an Assembled AVR Program

1. With the HEX file uploaded to the microcontroller, you can observe the behavior of the sample program. The LEDs on the ATmega32U4 board should indicate that the board is performing a basic BumpBot routine (see Figure 1 for a full description of the routine). To receive points for Part 1 of this lab, modify the sample program so the TekBot reverses for **twice as long** before turning away and resuming forward motion. **In your demo, explain how you have accomplished the aforementioned task.**

PROCEDURE - PART 2

Looking at C Code

- Initialize relevant I/O ports
- Loop forever:
 - Move forward for 500 ms
 - Move backward for 500 ms
 - Turn left for 1000 ms
 - Turn right for 2000 ms
 - Turn left for 1000 ms

Figure 2: Operations for the `DanceBot.c` code.

1. Download the sample C code (`DanceBot.c`). This simple C program is well-commented and ready to compile. Again, all the codes that you develop should be as well-commented as this code. The behavior of this program is described in Figure 2. Save this code to your favorite directory.
2. Examine the source file and try to understand how it is performing the operations described in Figure 2.

Your Own Code

Next, you need to write a simple C program that will make a TekBot perform the basic BumpBot routine, as seen and described in Part 1. The TekBot should travel forward until it encounters an object (that is, until one or both of the whiskers are bumped), back up and turn away from the object, and then resume moving forward. Here are a few tips for completing this task successfully:

- You should start from the *skeleton code* (`ece370-L1_skelton.c`) and add code to it as needed. Even if you do not use the skeleton code, you **must adhere** to the ‘PORT MAP’ given in the skeleton code when writing your whisker detection and motor control logic.
- If **both whiskers are triggered at the same time**, the TekBot must **back up and turn to the left** like it does when only the right whisker is hit.

- Don’t forget to make use of the `_delay_ms()` function.

When you have completed your C-based BumpBot program, demonstrate its correct operation to your lab TA to receive implementation credit for this lab.

LAB REPORT

You are **not** required to write a lab report, but you should review the study questions below. Although these questions will not be graded, they **can be asked during your demo, and your grade may be affected** if you cannot answer them.

Study Questions

1. Take a look at the code you downloaded for today’s lab. Notice the lines that begin with `.def` and `.equ` followed by some type of expression. These are known as **pre-compiler directives**. Define pre-compiler directive. What is the difference between the `.def` and `.equ` directives? (HINT: see Section 5.4 of the *AVR Assembler User Guide*).

2. Read the *AVR Instruction Set Manual*, and explain the AVR instructions listed below.

ADIW, BCLR, BRCC, BRGE, COM, EOR, LSL, LSR, NEG, OR, ORI, ROL, ROR, SBC, SBIW, and SUB.

3. The ATmega32U4 microcontroller has five general-purpose input-output (I/O) *ports*: Port B through Port F. An I/O port is a collection of *pins*, and these pins can be individually configured to send (output) or receive (input) a single binary bit. Each port has three I/O registers that are used to control the behavior of its pins: `PORTx`, `DDRx`, and `PINx`. (where “x” can be B-F; e.g., Port C’s three I/O registers are `PORTC`, `DDRC`, and `PINC`.)

- (a) Suppose you want to configure Port B so that all 8 of its pins are outputs. Which I/O register is used to accomplish this, and what 8-bit binary value must be written to configure all 8 pins as outputs?
- (b) Suppose Port D’s pins 4-7 have been configured as inputs. Which I/O register must be used to read the current state of Port D’s pins?
- (c) Does the function of a `PORTx` register differ depending on the setting of its corresponding `DDRx` register? If so, explain any differences.

4. This lab required you to modify the sample AVR program so the TekBot can reverse for twice as long before turning away and resuming forward motion. Clearly explain how you have accomplish this task.
5. The Part 2 of this lab required you to compile two C programs (one given as a sample, and another that you wrote) into a binary representation that allows them to run directly on your **ATmega32U4** board. Explain some of the benefits of writing code in a language like C that can be “cross compiled”. Also, explain some of the drawbacks of writing code this way.
6. The C program you wrote does basically the same thing as the sample AVR program you looked at in Part 1. What is the size (in bytes) of your Part 1 and Part 2 **output .hex files**? Explain why there is a size difference between these two files, even though they both perform the same BumpBot behavior?